

Proyecto: PRIMES is in P

Juan Camilo Solano, Nicolás Hernández Derch, Samuel Montoya Salazar

17 de Mayo, 2026

Abstract

En este documento buscamos hacer una demostración a cabalidad de la existencia de un algoritmo polinomial determinístico para determinar si un número es primo o no.

1 Historia de cómo encontrar números primos

A lo largo de la historia, los matemáticos han desarrollado un interés intrínseco por encontrar números primos. Sus propiedades abarcan todas las áreas de las matemáticas, llegando desde su origen la teoría de números hasta a áreas en el álgebra y geometría. Es por esto que desde los griegos, se busca una manera de encontrar primos de diferentes maneras.

Con el tiempo los matemáticos encontraron muchos métodos. El más antiguo y más sencillo de aplicar es el método de la criba de Eratóstenes, que consiste en calcular la raíz cuadrada del número del que se quiere si es primo o no. Si la raíz es un entero entonces el número era un cuadrado y por tanto no era primo, de lo contrario se calcula el piso de la raíz y miran los números primos menores a este número, si alguno de estos números divide al número quiere decir que el número no es primo.

Pero hay algo que todos estos métodos pasan por alto, y es que el número de números primos menores a un número n es aproximadamente $\frac{\sqrt{n}}{\log(\sqrt{n})}$, lo que hace que el método de la criba de Eratóstenes tenga una complejidad de $O(\frac{\sqrt{n}}{\log(\sqrt{n})})$ lo que no es de complejidad polinomial.

Lo que quiere decir esto es que, aunque el método de un resultado correcto, no es un método eficiente para encontrar números primos. Otro método que se usaba antes era el método usando el pequeño teorema de Fermat, que dice que si p es un número primo y a es un número entero que no es divisible por p , entonces $a^{p-1} \equiv 1 \pmod{p}$. Este método se creía que servía para encontrar primos pues el algoritmo sería elegir un número a y calcular $a^{n-1} \pmod{n}$, si el resultado es diferente de 1, entonces n no es primo. y se prueba con los a menores a n . Sin embargo, este método no es correcto pero sí polinomial a diferencia del anterior método, pues existen números compuestos llamados pseudoprimos de Fermat o los números de Carmichael que cumplen con esta propiedad para todos los valores de a con los que sean coprimos con n .

Si bien existen muchos criterios para encontrar números primos, como el test de Willson, la idea principal del algoritmo es que el criterio en el cual se basa el algoritmo tiene una cota lo suficientemente buena para lograr una complejidad polinomial menor a la que ya se tenía.

2 Algoritmo AKS

Para iniciar la demostración de este algoritmo, vamos a demostrar el criterio esencial el cual es muy similar al Pequeño teorema de Fermat, pero con una cota mucho más estricta.

Teorema.

Sea $a \in \mathbb{Z}, n \in \mathbb{N}, n \geq 2$ y $(a, n) = 1$, entonces n es primo si y solo si $(x + a)^n \equiv x^n + a \pmod{n(x)}$ $\square$

Demostración.

Para demostrar la implicacion directa suponga que n es primo entonces por teorema binomial se tiene que:

$$\begin{aligned}(x+a)^n &= \sum_{k=0}^n \binom{n}{k} x^{n-k} a^k \\ &= x^n + a^n \pmod{n(x)}\end{aligned}$$

pues cada termino de $\binom{n}{k}$ con $1 \leq k \leq n-1$ es divisible por n pues siempre esta el coeficiente de n en el numerador. Esto deja que $(x+a)^n \equiv x^n + a^n \pmod{n(x)}$ y por pequeño teorema de Fermat se tiene que $a^n \equiv a \pmod{n}$ y por tanto $(x+a)^n \equiv x^n + a \pmod{n(x)}$.

Para la suficiencia suponga que n es compuesto, entonces existe un primo q tal que $q|n$ y considere $k \in \mathbb{N}$ tal que $q^k || n$. Note ahora que $q^k \nmid \binom{n}{q}$ pues $\binom{n}{q} = \frac{n!}{q!(n-q)!} = \frac{(n-1)!m}{(q-1)!(n-q)!}$ dejando que $q^{k-1} | m \rightarrow q^k \nmid \binom{n}{q}$. Como $(a, n) = 1$ entonces $q^k \nmid a \rightarrow q^k \nmid a^r \forall r \in \mathbb{N}$, dejando que como $(q^k, a^{n-q}) = 1$ entonces el coeficiente en la expansion binomial del termino x^q no va a ser 0 modulo n y por tanto $(x+a)^n - x^n + aa \not\equiv 0 \pmod{n(x)}$ dando a lugar a una contradiccion

□

2.1 Notaciones

Defina el orden multiplicativo de n modulo r como el menor entero positivo k tal que $n^k \equiv 1 \pmod{r}$ equivalente a el orden en $\mathbb{Z}_r^*$ y se denota por $O_r(n)$, es importante aclarar que el orden solo tiene sentido si $(n, r) = 1$.

$\mathbb{Z}_n$ denota el anillo con elementos modulo n , y $\mathbb{F}_p$ denota el campo con p elementos, es decir $\mathbb{Z}_p$ con p primo. Si $h(x)$ es un polinomio de grado d entonces $\mathbb{F}[x]/(h(x))$ denota el cuerpo finito de orden p^d . Decimos que $f(x) \equiv g(x) \pmod{h(x), n}$ para representar que $f(x)$ y $g(x)$ son equivalentes en el anillo $\mathbb{Z}_n[x]/(h(x))$.

$\phi(n)$ denota la funcion de Euler totient, es decir el numero de enteros positivos menores a n que son coprimos con n .

Para explicar la complejidad temporal del algoritmo, usaremos el símbolo $\tilde{O}(t(n))$ para denotar $O(t(n)) \cdot \text{poly}(\log t(n))$, donde $t(n)$ es cualquier función de n . Por ejemplo, $\tilde{O}(\log^k n) = O(\log^k n \cdot \text{poly}(\log \log n)) = O(\log^{k+\varepsilon} n)$ para cualquier $\varepsilon > 0$. Usaremos $\log$ para logaritmo en base 2, y $\ln$ para el logaritmo natural.

Ahora con este resultado podemos empezar a entrar mas a profundidad en el algoritmo. Si bien es un resultado muy importante en las matematicas su importancia no es proporcional con su dificultad a diferencia de muchas otras cosas dentro del area. Es un poco contraintuitivo que un resultado asi se demorase tanto en ser descubierto puesto que el conocimiento de aqui en adelante para demostrar el algoritmo es un conocimiento de pregrado de matematicas y posiblemente menos para ser demostrado en su totalidad. A tal efecto que quienes lo demostraron fueron estudiantes.

2.2 El Algoritmo

Entrada: entero $n > 1$.

1. Si $(n = a^b)$ Para $a \in \mathbb{N}$ y $b > 1$, retorne **Compuesto**.
2. Encuentre el r mas pequeño tal que $\phi(r) > \log^2 n$.
3. Si $1 < (a, n) < n$ Para algun $a \leq r$, retorne **Compuesto**.
4. Si $n \leq r$, retorne **Primo**.¹
5. Desde $a = 1$ hasta $\lfloor \sqrt{\phi(r)} \log n \rfloor$ haga lo siguiente:
 Si $((X + a)^n \neq X^n + a \pmod{X^r - 1, n})$, retorne **Compuesto**;
6. retorne **Primo**.

Ahora vamos a empezar a demostrar los resultados necesarios que nos llevaran al resultado final.

2.3 Demostracion del Algoritmo

Lema.

Sea $L(m) = \text{mcm}(1, 2, \dots, m)$. Para todo $m \geq 7$, se cumple que:

$$L(m) \geq 2^m$$

□

Demostración.

La demostración se basa en el estudio de la integral: $I_n = \int_0^1 x^n (1-x)^n dx$ Es un resultado conocido que $L(2n+1)I_n$ es un entero positivo. Dado que $I_n = \frac{(n!)^2}{(2n+1)!}$, tenemos:

$$L(2n+1) \geq \frac{(2n+1)!}{(n!)^2} = (2n+1) \binom{2n}{n} \quad (1)$$

Utilizando la desigualdad para el coeficiente binomial central $\binom{2n}{n} \geq \frac{4^n}{2n}$, para $n \geq 1$, se obtiene:

$$L(2n+1) \geq (2n+1) \frac{4^n}{2n} = \left(1 + \frac{1}{2n}\right) 2^{2n}.$$

Para $m = 2n+1$, esto implica $L(m) > 2^{m-1}$. Un refinamiento de este método mediante el uso de integrales de la forma $\int_0^1 x^{n+k} (1-x)^n dx$ permite fortalecer la cota.

Específicamente, para $m \geq 7$, procedemos por inducción o verificación directa. Para el caso base $m = 7$:

$$L(7) = 420 > 2^7 = 128$$

Para el paso inductivo, se utiliza el hecho de que $L(m+1) = L(m) \cdot \frac{m+1}{\text{mcd}(L(m), m+1)}$. El crecimiento de la función $\psi(m) = \ln L(m)$ está acotado inferiormente por $m \ln 2$ para $m \geq 7$ según el análisis de las integrales de Chebyshev, lo que concluye que:

$$L(m) \geq 2^m$$

□

Ahora vamos a demostrar las dos implicaciones del teorema que van a terminar dando el resultado final. La primera de estas es mucho mas facil que la reciproca y por ello se siguen los siguientes resultados.

Teorema.

Si n es primo entonces el algoritmo retorna **Primo**.

□

Demostración.

Como n es primo entonces no se cumple ni el paso 1 ni el 3 por definicion de ser primo. Ahora por el teorema 1 se tiene que n no puede cumplir el paso 5, pues si n es primo entonces se cumple que $(x+a)^n \equiv x^n + a \pmod{n(x)}$ para todo a , y por tanto el algoritmo retorna **Primo** sin importar en que paso lo retorne. $\square$

De ahora en adelante el logaritmo siempre se va a tomar en base 2 a menos que se indique lo contrario

Lema.

$\exists r \leq \max\{3, \lceil \log^5 n \rceil\}$ tal que $O_r(n) > \log^2 n$. $\square$

Este resultado nos es util pues para el paso 2 del algoritmo necesitamos encontrar este r y es importante que este r no sea tan grande para que el algoritmo siga siendo polinomial.

Demostración.

Note que para $n = 2, r = 3$ se cumple la condicion trivialmente pues $\max\{3, \lceil \log^5 n \rceil\} = 3$ y $O_3(2) = 2 > 1 = \log^2(2)$. Suponga que $n > 2$ entonces como $\log n \geq \log 3 \geq \sqrt[5]{10}$ Entonces $\log^5 n \geq 10$ y $\lceil \log^5 n \rceil \geq 10$. Considere $r_1, r_2, \dots, r_t$ tales que $O_{r_i}(n) \leq \log^2 n$ o $r_i | n$, entonces todos los r_i dividen al siguiente producto:

$$n \prod_{i=1}^{\lfloor \log^2 n \rfloor} (n^i - 1) \quad (2)$$

puesto que si $r_i | n$ se tiene trivialmente, y si $O_{r_i}(n) \leq \log^2 n$ entonces el orden de r_i debe estar en el intervalo de $[1, \log^2 n]$ entonces, en particular, estan todos los ordenes de todos los r_i que no dividen a n . Por otro lado,

$$P = n \prod_{i=1}^{\lfloor \log^2 n \rfloor} (n^i - 1) \leq n^{\log^4 n} \leq 2^{\log^5 n} \quad (3)$$

Dado que $n^i \leq n^{\lfloor \log^2 n \rfloor}$ y

$$n \prod_{i=1}^{\lfloor \log^2 n \rfloor} (n^i - 1) \leq n \prod_{i=1}^{\lfloor \log^2 n \rfloor} n^i \leq n^{2 \lfloor \log^2 n \rfloor} \leq n^{2 \log^2 n} = n^{\log^4 n} \quad (4)$$

ahora por el lema 1 se tiene que $L(\lceil \log^5 n \rceil) \geq 2^{\lceil \log^5 n \rceil} > 2^{\log^5 n} = n^{\log^4 n}$, lo que implica que $n \prod_{i=1}^{\lfloor \log^2 n \rfloor} (n^i - 1) < L(\lceil \log^5 n \rceil)$, lo que implica que debe existir un $s \leq \lceil \log^5 n \rceil$ tal que s no es un r_i , es decir, $O_s(n) > \log^2 n$ y s no divide a n . Ahora si $(s, n) = 1$ queda demostrado el lema, si $(s, n) > 1$ entonces $s | n$ o $O_s(n) \leq \log^2 n$. Defina $r = \frac{s}{(s, n)}$ veamos que $r \notin \{r_1, r_2, \dots, r_t\}$, pues si $r = r_i$ entonces hay dos casos, $r | P$ o $O_{r_i}(n) \leq \log^2 n$. Si $O_{r_i}(n) \leq \log^2 n$ y $(r_i, n) = 1$ entonces defina $d = (s, n)$, entonces $(r, d) = 1$ pues tanto r como d dividen a P entonces como tienen minimo comun divisor 1 entonces $rd | P$ lo que es una contradiccion pues $s = r(s, n) | P$ lo que es una contradiccion pues s no es un r_i . Ahora si $r | n$ entonces veamos que $(r, d) = 1$, suponga que no entonces existe un primo q tal que $q | r$ y $q | d$, pero como $q | s, q | n$ entonces $qd | s$ y $qd | n$ dejando asi que $qd | d$ entonces $q = 1$ contradiccion entonces como $(r, d) = 1$ entonces por el argumento de antes queda que $s | n$ contradiccion. $\square$

Con este lema podemos ir avanzando en los pasos del algoritmo, sabemos que el paso 1 ya no se cumple pues de lo contrario claramente habria retornado compuesto. Supongamos que el paso 1 no dio como resultado compuesto, ahora por el anterior lema podemos ejecutar el paso 2 sin problemas pues sabemos que existe un r . Ahora como sabemos que $O_r(n) > 1$ entonces por Teorema de Lagrange debe existir un divisor primo de n , p , tal que $O_r(p) > 1$. Ahora $p > r$ pues si $p \leq r$ entonces el paso 3 del algoritmo hubiera retornado compuesto, por otro lado, si $n \leq p$ entonces esto deberia retornar primo pues tenemos un numero cuyo orden no es potencia de ninguno de los anteriores en un $\mathbb{Z}_r^*$ que es un grupo donde n tiene orden $O_r(n)$ talque $O_r(p) | O_r(n)$ pero $O_r(p)$ es el orden del grupo entonces n tiene que ser primo obligatoriamente. Ahora tenemos que $p \geq r$ y por el paso 2 que $(n, r) = 1$, de ahora en adelante tanto p como r van a estar

figos. Sea $l = \lfloor \sqrt{\phi(r)} \log n \rfloor$, el algoritmo ahora en el paso 5 va a verificar l ecuaciones. Suponga que el algoritmo no retorna compuesto en el paso 5 entonces tenemos que

$$(x + a)^n \equiv x^n + a \pmod{(x^r - 1, n)}, \forall a \in [0, l] \quad (5)$$

Pero como $p|n$ entonces tenemos que

$$(x + a)^n \equiv x^n + a \pmod{(x^r - 1, p)}, \forall a \in [0, l] \quad (6)$$

Ahora por el teorema 1 se tiene que

$$(x + a)^p \equiv x^p + a \pmod{(x^r - 1, p)}, \forall a \in [0, l] \quad (7)$$

Como en un cuerpo de caracterisitica p , elevar a la p es inyectivo, pues

$$f(x) = g(x) \rightarrow f(x) - g(x) = 0 \rightarrow (f(x) - g(x))^p = 0 \rightarrow f(x)^p - g(x)^p = 0 \rightarrow f(x)^p = g(x)^p \pmod{p} \quad (8)$$

Entonces por las ecuaciones anteriores note que

$$(x + a)^{(n/p)p} = (x + a)^n = x^n + a = x^{(n/p)p} + a = (x^{n/p} + a)^p \pmod{(x^r - 1, p)}, \forall a \in [0, l] \quad (9)$$

Dejando que

$$(x + a)^{(n/p)p} \equiv x^{(n/p)p} + a \pmod{(x^r - 1, p)}, \forall a \in [0, l] \quad (10)$$

Y como sabemos que elevar a la p es inyectivo queda el $(x + a)^{(n/p)} \equiv x^{(n/p)} + a \pmod{(x^r - 1, p)}, \forall a \in [0, l]$ Esta propiedad la vamos a llamar introspectividad y va a ser importante mas adelante.

Definición.

Para un polinomio $f(x)$ y un numero $m \in \mathbb{N}$, se dice que $f(x)$ es introspectivo para m si $f(x)^m \equiv f(x^m) \pmod{(x^r - 1, p)}$. $\square$

Ahora mostramos dos propiedades de números introspectivos que nos servirán más adelante:

Lema.

Si m, n son enteros introspectivos para $f(x)$ entonces mn es introspectivo para $f(x)$. $\square$

Demostración.

Note que $f(x)^{mn} = (f(x)^m)^n \equiv f(x^m)^n \pmod{(x^r - 1, p)}$, reemplazando $x^m = y$ tenemos que $f(y)^n \equiv f(y^n) \pmod{(x^r - 1, p)}$, reemplazando $y = x^m$ tenemos que $f(x^m)^n \equiv f((x^m)^n) \pmod{(x^r - 1, p)}$, lo que implica que $f(x)^{mn} \equiv f(x^{mn}) \pmod{(x^r - 1, p)}$ y por tanto mn es introspectivo para $f(x)$. $\square$

Lema.

Si m es introspectivo para $f(x)$ y para $g(x)$ entonces lo es para $f(x)g(x)$. $\square$

Demostración.

Tenemos que

$$[f(x)g(x)]^m = [f(x)^m][g(x)^m] \equiv f(x^m)g(x^m) \pmod{(x^r - 1, p)} \quad (11)$$

$\square$

Con lo anterior podemos hacer las siguientes observaciones:

- Por Lema, tenemos que $\frac{n}{p} \cdot \frac{n}{p}$ y $p \cdot p$ son ambos introspectivos para $x + a$ para $0 \leq a \leq l$. Así, $I = \{(n/p)^i \cdot p^j | i, j \in \mathbb{N}\}$ es un conjunto de enteros introspectivos para $x + a$ para $0 \leq a \leq l$.
- Para todo $m \in I$ tenemos que m es introspectivo para $(x + a)^{e_a}$ con $e_a \geq 0$, por lo que todo $m \in I$ es introspectivo para todo polinomimo en $P = \left\{ \prod_{a=0}^{\ell} (X + a)^{e_a} \mid e_a \geq 0 \right\}$.

Procedemos a definir dos grupos cuyas cardinalidades nos ayudarán a mostrar la validez del algoritmo:

- $G = \{m \bmod r \mid m \in I\}$ subgrupo de $\mathbb{Z}_r^*$, pues $\gcd(n, r) = \gcd(p, r) = 1$. Tome $t := |G|$. Note que $G = \langle \bar{n}, \bar{p} \rangle$. Como $o_r(n) > \log^2 n$ y $o_r(n) \mid t$ (por Lagrange) entonces $t > \log^2 n$
- Sea $Q_r(x)$ el r -ésimo polinomio ciclotómico sobre $\mathbb{F}_p$. Por definición, $Q_r(x) \mid x^r - 1$ y por propiedades de polinomios ciclotómicos, $Q_r(x)$ se puede factorizar en factores irreducibles de grado $o_r(p) > 1$. Sea $h(x)$ uno de esos factores. Entonces $\deg(h(x)) = o_r(p) \geq 2$. Definimos

$$\mathcal{G} = \{\overline{p(x)} \in F \mid p(x) \in P\} \leq F := \mathbb{F}_p[x]/(h(x)) \quad (12)$$

generado por $\bar{x}, \overline{x+1}, \dots, \overline{x+l} \in F$

Teniendo estos grupos, podemos encontrar una cota inferior al tamaño de $\mathcal{G}$ utilizando t :

Lema.

$$|\mathcal{G}| \geq \binom{t+l}{t-1}$$

□

Demostración.

Como $h(x)$ es un factor de $Q_r(x)$ que es ciclotómico, entonces x es raíz r -ésima primitiva de la unidad en F ya que

$$Q_r(x) \mid x^r - 1 \rightarrow h(x) \mid x^r - 1 \rightarrow \overline{x^r - 1} = \bar{0} \text{ en } F \quad (13)$$

Sean $f(x), g(x)$ polinomios tal que $\deg(f(x)), \deg(g(x)) < t$. Queremos ver que si $f(x) \neq g(x)$ entonces $f(x) \neq g(x)$. Suponga que $f(x) = g(x)$ en F . Para $m \in I$ tenemos que $[f(x)]^m = [g(x)]^m$ en F . Como $h(x) \mid x^r - 1$, entonces $f(x^m) = g(x^m)$. Como tomamos m arbitrario tenemos que $\forall m \in I : \bar{x}^m$ es raíz de $Q(y) := f(y) - g(y)$. Como $\gcd(m, r) = 1$, entonces $\bar{x}^m$ es r -ésima raíz primitiva de la unidad. Así, hay $|G| = t$ raíces distintas de $Q(y)$ en F ; pero $\deg(Q(y)) < t$ lo que es una contradicción. Debe ocurrir que $f(x) = g(x)$ en F . Ahora, como $l = \lfloor \sqrt{\phi(r)} \log n \rfloor < \lfloor \sqrt{r} \log n \rfloor \leq \sqrt{r} \log n < r$ (la última desigualdad ya que $o_r(n) \leq \phi(r)$) y $\phi(r) < r$, entonces $\log^2 n < o_r(n) \leq \phi(r) < r$ por lo que $l < rr$ y $p > r$. Tenemos que si $1 \leq i \neq j \leq l$ se tiene que $\bar{i} \neq \bar{j}$ en $\mathbb{F}_p$. Obtenemos que $\bar{x}, \overline{x+1}, \dots, \overline{x+l}$ son distintos en F . Como $\deg(h(x)) \geq 2$, entonces $\overline{x+a} \neq \bar{0}$ para $0 \leq a \leq l$. Concluimos que existen $l+1$ polinomios distintos de grado $< t$ en $\mathcal{G}$ y así existen al menos $\binom{t+(l+1)-1}{t-1} = \binom{t+l}{t-1}$ polinomios distintos de grado $< t$ en $\mathcal{G}$. □

Seguimos con un último lema que nos ayudará con la caracterización de n :

Lema.

Si n no es potencia de p , entonces $|\mathcal{G}| \leq n^{\sqrt{t}}$

□

Demostración.

Sea $\hat{I} = \left\{ \left(\frac{n}{p} \right)^i p^j \mid 0 \leq i, j \leq \lfloor \sqrt{t} \rfloor \right\} \subseteq I$. Note que $\forall i, j : \left(\frac{n}{p} \right)^i p^j$ es único por lo que $|\hat{I}| = (\lfloor \sqrt{t} \rfloor + 1)^2 > t$.

Deben existir $m_1, m_2 \in \hat{I}$ tal que $m_1 \equiv m_2 \pmod{r}$. Sin pérdida de la generalidad, suponga que $m_1 > m_2$. Tenemos que $x^{m_1} \equiv x^{m_2} \pmod{x^r - 1}$ Sea $f(x) \in P$. Entonces

$$[f(x)]^{m_1} \equiv f(x^{m_1}) \equiv f(x^{m_2}) \equiv [f(x)]^{m_2} \pmod{x^r - 1} \rightarrow \overline{[f(x)]^{m_1}} = \overline{[f(x)]^{m_2}} \text{ en } F \quad (14)$$

Así, $\overline{f(x)} \in \mathcal{G}$ es raíz de $Q'(y) := \overline{y^{m_1} - y^{m_2}}$ en F . Como tomamos $f(x)$ arbitrario, entonces $Q'(y)$ tiene al menos $|\mathcal{G}|$ raíces distintas en F . Además, $\deg(Q'(y)) = m_1$ por construcción, y tenemos que $\exists i, j : m_1 = \left(\frac{n}{p} \right)^i p^j \leq \left(\frac{n}{p} \right)^{\lfloor \sqrt{t} \rfloor} p^{\lfloor \sqrt{t} \rfloor} = n^{\lfloor \sqrt{t} \rfloor} \leq n^{\sqrt{t}}$. Como $|\mathcal{G}| \leq m_1$, entonces $|\mathcal{G}| \leq n^{\sqrt{t}}$. □

Con todos los resultados anteriores, podemos demostrar la implicación recíproca del teorema 1 y mostrar la validez del algoritmo.

Teorema.

Si el algoritmo retorna **Primo** entonces n es primo.

□

Demostración.

Suponga que el algoritmo retorna **Primo**. Por el lema anterior, para $|G| = t$ y

$$l = \left\lfloor \sqrt{\phi(r)} \log n \right\rfloor,$$

se tiene que

$$|\mathcal{G}| \geq \binom{t+l}{t-1} \quad (15)$$

$$\geq \binom{l+1+\lfloor \sqrt{t} \log n \rfloor}{\lfloor \sqrt{t} \log n \rfloor}, \quad (16)$$

ya que

$$t \geq o_r(n) > \log^2 n \rightarrow \sqrt{t} > \log n \rightarrow t > \sqrt{t} \log n \rightarrow t > \lfloor \sqrt{t} \log n \rfloor.$$

Además,

$$\binom{l+1+\lfloor \sqrt{t} \log n \rfloor}{\lfloor \sqrt{t} \log n \rfloor} \geq \binom{2\lfloor \sqrt{t} \log n \rfloor + 1}{\lfloor \sqrt{t} \log n \rfloor}, \quad (17)$$

ya que, como $G \leq \mathbb{Z}_r^*$, se tiene que

$$t \mid \phi(r) \rightarrow t \leq \phi(r) \rightarrow l = \lfloor \sqrt{\phi(r)} \log n \rfloor \geq \lfloor \sqrt{t} \log n \rfloor.$$

Finalmente,

$$\binom{2\lfloor \sqrt{t} \log n \rfloor + 1}{\lfloor \sqrt{t} \log n \rfloor} \geq 2^{\lfloor \sqrt{t} \log n \rfloor + 1} \quad (18)$$

$$\geq 2^{\sqrt{t} \log n} = n^{\sqrt{t}}. \quad (19)$$

Por el lema anterior, $n = p^\alpha$ para algún $\alpha \in \mathbb{N}^+$. Si $\alpha > 1$, el algoritmo lo hubiera detectado en el paso 1. Por lo tanto, debe ocurrir que $n = p$. $\square$

2.4 Complejidad temporal del algoritmo

Para calcular la complejidad temporal del algoritmo, tendremos en cuenta que las operaciones aritméticas básicas (suma, multiplicación y división) entre dos números de m bits requieren tiempo $\tilde{O}(m)$ [2]. Esto se debe a que existen algoritmos eficientes de multiplicación, como el de Karatsuba, que realizan estas operaciones en tiempo casi lineal respecto al número de bits. De manera análoga, si trabajamos con polinomios de grado d cuyos coeficientes tienen a lo sumo m bits, las mismas operaciones toman tiempo $\tilde{O}(d \cdot m)$ [2], pues ahora debemos operar sobre d coeficientes, cada uno de m bits, lo que multiplica el costo por un factor de d .

Teorema.

La complejidad temporal del algoritmo es $\tilde{O}(\log^{21/2} n)$. $\square$

Demostración.

Para hallar la complejidad temporal del algoritmo, analizaremos cada paso por separado.

Para el paso 1, debemos verificar si $n = a^b$ para algunos naturales a, b con $b \geq 2$. Primero, notemos que b no puede ser arbitrariamente grande: como $a \geq 2$, se tiene $2^b \leq a^b = n$, de donde $b \leq \log n$. Así, basta probar todos los valores de b entre 2 y $\lfloor \log n \rfloor$, lo cual toma tiempo $O(\log n)$.

Fijado un valor de b , queremos determinar si existe un entero a tal que $a = n^{1/b}$. Esto se realiza mediante búsqueda binaria en el intervalo $[2, \sqrt{n}]$. Dado que n tiene $O(\log n)$ bits, la búsqueda binaria realiza $O(\log n)$ iteraciones. En cada iteración se debe calcular a^b y compararlo con n . Usando multiplicaciones sucesivas, calcular a^b requiere $O(\log b) = O(\log \log n)$ multiplicaciones de números de a lo sumo $\log n$ bits, y como cada multiplicación toma tiempo $\tilde{O}(\log n)$, el cálculo de a^b toma tiempo $\tilde{O}(\log n)$. Por lo tanto, cada iteración de la búsqueda binaria cuesta $\tilde{O}(\log n)$, y como hay $O(\log n)$ iteraciones, el costo total por cada valor de b es $\tilde{O}(\log^2 n)$.

Combinando ambos factores, el costo total del paso 1 es $O(\log n) \cdot \tilde{O}(\log^2 n) = \tilde{O}(\log^3 n)$ [2].

En el paso 2, buscamos un r tal que $o_r(n) > \log^2 n$. Esto se hace probando valores sucesivos de r y verificando que $n^k \not\equiv 1 \pmod{r}$ para todo $k \leq \log^2 n$. Para un r particular, esto requiere a lo sumo

$O(\log^2 n)$ multiplicaciones módulo r , por lo que toma tiempo $\tilde{O}(\log^2 n \cdot \log r)$. Por el Lema 4.3, sabemos que solo es necesario probar $O(\log^5 n)$ valores distintos de r . Por lo tanto, la complejidad total del paso 2 es $\tilde{O}(\log^7 n)$.

El paso 3 consiste en calcular el mcd de a y n , donde a es algun natural menor o igual que r . Esto corresponde a calcular r veces un mcd. Cada cálculo de mcd toma tiempo $O(\log n)$ [2] usando el Algoritmo de Euclides, y por lo tanto la complejidad temporal de este paso es $O(r \log n) = O(\log^6 n)$.

La complejidad temporal del paso 4 es simplemente $O(\log n)$, pues solo se compara n con r .

En el paso 5, debemos verificar $\lfloor \sqrt{\phi(r)} \log n \rfloor$ ecuaciones, pues ese es el número de valores de a que se prueban. Cada ecuación es de la forma $(X + a)^n \equiv X^n + a \pmod{X^r - 1, n}$. Analicemos el costo de verificar cada una de estas ecuaciones.

Cada verificación consiste en calcular el lado izquierdo $(X + a)^n$ en el anillo $\mathbb{Z}_n[X]/(X^r - 1)$. Esto se hace mediante exponenciación rápida, que requiere $O(\log n)$ multiplicaciones de polinomios. Cada polinomio en este anillo tiene grado a lo sumo $r - 1$ y coeficientes de tamaño $O(\log n)$ bits. Por lo discutido anteriormente, multiplicar dos polinomios de grado d con coeficientes de m bits toma tiempo $\tilde{O}(d \cdot m)$. En nuestro caso, $d = r$ y $m = \log n$, por lo que cada multiplicación de polinomios cuesta $\tilde{O}(r \log n)$. Realizando $O(\log n)$ de estas multiplicaciones, el costo de verificar cada ecuación es $\tilde{O}(r \log n) \cdot O(\log n) = \tilde{O}(r \log^2 n)$.

Finalmente, dado que hay $\lfloor \sqrt{\phi(r)} \log n \rfloor$ ecuaciones que verificar, el costo total del paso 5 es:

$$\tilde{O}(r \log^2 n) \cdot O\left(\sqrt{\phi(r)} \log n\right) = \tilde{O}\left(r \sqrt{\phi(r)} \log^3 n\right).$$

Como $\phi(r) \leq r$ y, por el Lema 4.3, $r = O(\log^5 n)$, tenemos $r \sqrt{\phi(r)} \leq r^{3/2} = O(\log^{15/2} n)$, y por tanto el costo total es $\tilde{O}(\log^{15/2} n \cdot \log^3 n) = \tilde{O}(\log^{21/2} n)$. El tiempo de este paso domina a todos los demás. Por tanto, concluimos que la complejidad temporal del algoritmo es $\tilde{O}(\log^{21/2} n)$. $\square$

3 Aplicaciones computacionales

Este algoritmo, conocido como el algoritmo AKS debido a las iniciales de los nombres de los creadores, representa un hito fundamental en la teoría de la complejidad computacional y la teoría de números. En esta sección exponemos sus principales aplicaciones e implicaciones, así como sus limitaciones en contextos prácticos.

3.1 Criptografía

Uno de los campos donde la verificación de primalidad tiene mayor relevancia práctica es la criptografía, en particular el criptosistema RSA. En este esquema, la seguridad del sistema depende de la dificultad de factorizar el producto $n = p \cdot q$ de dos números primos grandes p y q . Para generar las claves, es necesario producir primos del orden de cientos o miles de dígitos decimales, lo cual exige un método confiable y eficiente para verificar la primalidad de un candidato [1].

Antes de que saliera el algoritmo AKS, los algoritmos utilizados en la práctica para este fin eran probabilísticos, como el test de Miller-Rabin [3]. Estos algoritmos son muy eficientes, pero tienen una pequeña probabilidad de error: pueden clasificar erróneamente un número compuesto como primo. Si en RSA se generara un número compuesto en lugar de un primo, la clave privada se generaría incorrectamente, haciendo que el descifrado sea prácticamente imposible, o peor aún, si la clave pública resultara fácilmente factorizable, la seguridad del sistema quedaría comprometida. El algoritmo AKS, al ser determinista, elimina por completo esta posibilidad de error.

3.2 Limitaciones prácticas

A pesar de su importancia teórica, el algoritmo AKS presenta serias limitaciones cuando se intenta aplicar en contextos reales. La complejidad temporal del algoritmo original es $\tilde{O}(\log^{21/2} n)$ [1], y aunque versiones mejoradas la reducen a $\tilde{O}(\log^6 n)$, esto sigue siendo considerablemente más lento que los algoritmos probabilísticos usados en la práctica, como Miller-Rabin [3], cuya complejidad es $O(k \log^3 n)$, donde k es el número de iteraciones.

Además, el costo en memoria del algoritmo es demasiado alto para entradas de tamaño criptográfico. Dado que el algoritmo opera en el anillo $\mathbb{Z}_n[X]/(X^r - 1)$ con $r = O(\log^5 n)$, los polinomios involucrados tienen grado hasta $r - 1$, con coeficientes que pueden tener hasta $O(\log n)$ bits. Para un número n de 1024 bits, esto se traduce en requerimientos de almacenamiento del orden de 10^9 GB, lo cual es completamente inviable en la práctica [5].

Por estas razones, en aplicaciones criptográficas reales se prefieren los tests probabilísticos como Miller-Rabin, que ofrecen un equilibrio mucho más favorable entre velocidad y confiabilidad [3]. La probabilidad de error del test de Miller-Rabin, por ejemplo, es a lo sumo 4^{-k} tras k iteraciones, lo que en la práctica se vuelve astronómicamente pequeña con un número moderado de rondas.

3.3 Importancia teórica: PRIMES $\in$ P

Más allá de sus aplicaciones directas, el verdadero impacto del algoritmo AKS se refleja en la teoría de la complejidad computacional. Antes de 2002, que fue cuando se publicó el algoritmo AKS, era un problema abierto determinar si la verificación de primalidad podía resolverse en tiempo polinomial de manera determinista e incondicional. Los algoritmos deterministas existentes, como el test de Miller bajo la hipótesis de Riemann generalizada [4], requerían asumir conjeturas no demostradas para garantizar su corrección. Los algoritmos probabilísticos, por su parte, no ofrecían certeza absoluta.

El algoritmo AKS resolvió este problema de forma definitiva al demostrar que PRIMES $\in$ P, es decir, que el problema de decisión de determinar si un número dado es primo pertenece a la clase de complejidad P: la clase de problemas que se pueden solucionar en tiempo polinomial de manera determinista. Este resultado es incondicional, funciona para cualquier entero y no depende de ninguna hipótesis no demostrada.

Este logro, además de cerrar un problema abierto, abrió nuevas líneas de investigación en teoría de números computacional e impulsó el desarrollo de algoritmos más eficientes para problemas relacionados, consolidando la primalidad como uno de los problemas mejor comprendidos dentro de P.

References

- [1] M. Agrawal, N. Kayal y N. Saxena, *PRIMES is in P*, *Annals of Mathematics*, vol. 160, núm. 2, pp. 781–793, 2004. DOI: 10.4007/annals.2004.160.781
- [2] J. von zur Gathen y J. Gerhard, *Modern Computer Algebra*, Cambridge University Press, 1999.
- [3] M. O. Rabin, *Probabilistic algorithm for testing primality*, *Journal of Number Theory*, vol. 12, núm. 1, pp. 128–138, 1980. DOI: 10.1016/0022-314X(80)90084-0
- [4] G. L. Miller, *Riemann’s hypothesis and tests for primality*, *Journal of Computer and System Sciences*, vol. 13, núm. 3, pp. 300–317, 1976. DOI: 10.1016/S0022-0000(76)80043-8
- [5] Z. Cao, *Remarks on AKS Primality Testing Algorithm and a Flaw in the Definition of P*, *arXiv preprint*, arXiv:1402.0146, 2014. Disponible en: <https://arxiv.org/abs/1402.0146>